Foundational concepts of machine learning

Lecture Notes

Giorgio Gambosi

Master's Degree in Computer Science

University of Rome "Tor Vergata"

A.Y. 2026–2027

Contents

1 Introductory concepts **1**

1.1 Three paradigms of machine learning . . . 1
1.2 Data representation . . . 2
1.3 The learning process and generalization . . . 2
1.4 Formal Definition of a Supervised Machine Learning Task . . . 2
1.5 Relations between items and their targets . . . 3
1.6 Functional predictors . . . 3
1.7 Probabilistic predictors . . . 5

2 Measuring the quality of a predictor: the loss function **6**

2.1 Loss function definition, per-element risk, and expected risk . . . 7
2.2 The Empirical Risk as an Estimator of the Expected Risk . . . 8
2.3 Minimization of the empirical risk and its issues . . . 10
2.4 Bayes Predictors and Bayes Risk . . . 15

3 The bias-variance decomposition **16**

3.1 The general decomposition . . . 16
3.2 Interpretability, and relation with the error decomposition . . . 17

1 Introductory concepts

Machine learning rests on a simple observation: data is rarely random. Whether you analyse images, text, sensor measurements, or human behaviour, the data you collect contains patterns reflecting the underlying process that generated it. The core goal of machine learning is to find these patterns from observations, without explicitly programming the rules governing the data.

From a collection of observations you construct **models**: mathematical abstractions capturing the relationships present in the data, used for prediction and decision-making on new, unseen data.

1.1 Three paradigms of machine learning

1.1.1 Supervised Learning

In supervised learning you work with paired data: observations (as feature sets) and corresponding target values. The goal is to learn the relationship between observations and targets in order to predict targets for new inputs. For instance, given houses described by square footage, number of

bedrooms, and location, each with a known selling price, you learn a function mapping features to prices and use it to estimate the price of a new house.

Supervised learning splits into two categories depending on the nature of the target:

Regression The target is a real number (continuous): prices, temperatures, stock values, or any quantity on a numerical spectrum.

Classification The target is a discrete category: spam detection, handwritten digit recognition, diagnosis, sentiment analysis.

In both cases, we define a model: either a function $y(\mathbf{x})$ outputting a prediction directly, or a probability distribution $p(t|\mathbf{x})$ quantifying uncertainty about the target value. Training data are then used to adjust the model's parameters.

1.1.2 Unsupervised Learning

Unsupervised learning works without explicit targets, looking for structure in unlabelled data: clustering groups similar items; density estimation models the distribution generating the data; dimensionality reduction projects data onto lower-dimensional representations preserving key information (**Principal Component Analysis (PCA)**, for instance, retains the top k orthogonal directions of greatest variance).

In generative modelling, as in large language models, diffusion models and GANs, one learns to produce new synthetic data from the same process as the training data.

1.1.3 Reinforcement Learning

In reinforcement learning, rather than working with a fixed dataset, an agent learns through interaction with an environment, repeatedly choosing actions, observing the consequences, and adjusting its behaviour to maximize cumulative reward. At each step the agent observes the current state and selects an action; the environment returns a reward and a new state. These signals guide the agent toward an optimal policy $\pi : S \to A$, mapping states to actions, which maximizes expected cumulative reward:

$$\pi^* = \underset{\pi}{\operatorname{argmax}}\, \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \tag{1}$$

where $\gamma \in [0, 1]$ is a discount factor balancing immediate versus future rewards, and r_t is the reward at step t.

Its central tension is the exploration-exploitation tradeoff between trying new actions and exploiting actions known to be good. Since this framework differs substantially from the other two, these notes set RL aside.

1.2 Data representation

Every machine learning approach starts with a representation choice: real-world entities (images, documents, customers, transactions, biological samples) must be converted into feature vectors. A feature vector $\mathbf{x} = (x_1, x_2, \ldots, x_d)$ is a collection of d numerical measurements describing an object, and its choice affects both prediction quality and computational efficiency.

In image classification, for instance, raw pixel intensities yield millions of noisy features, while edge orientations or textures may be more informative. This is why feature engineering (combining, selecting or learning features) is often as important as the choice of learning algorithm.

1.3 The learning process and generalization

Across all paradigms you start with a **training set** $\mathcal{T} = (\mathbf{x}, \mathbf{t})$ containing n examples. In supervised learning, each example pairs an input feature vector $\mathbf{x}_i$ with its target value t_i; in unsupervised learning, you have only the feature vectors $\mathbf{x} = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$.

What matters is not fitting the training data but how well the model *generalizes* to unseen data: this is the core challenge of **generalization**. A model memorising the training data fails on new examples, an overly simple one misses important patterns. The **capacity** of a model determines its flexibility, the **bias-variance tradeoff** reflects the tension between underfitting and overfitting, and **regularisation** penalises complexity to improve generalization.

1.4 Formal Definition of a Supervised Machine Learning Task

A supervised learning task involves two key sets: the **domain set** $\mathcal{X}$ and the **label set** $\mathcal{Y}$.

The domain set $\mathcal{X}$ collects the objects to be labelled. Each object $\mathbf{x} \in \mathcal{X}$ is modelled as an array of **features**: $\mathbf{x} = (x_1, x_2, \ldots, x_d)$, where d is the **dimensionality** (more complex structures, such as sequences or graphs, are also possible).

The label set $\mathcal{Y}$ contains all possible output values. If $\mathcal{Y}$ is continuous, we face a **regression** task; when $\mathcal{Y}$ is discrete, a **classification** task: **binary classification** when $|\mathcal{Y}| = 2$ and **multi-class classification** when $|\mathcal{Y}| > 2$.

The learning algorithm $\mathcal{A}$ receives a **training set** $\mathcal{T} = \{(\mathbf{x}_1, t_1), \ldots, (\mathbf{x}_n, t_n)\}$, organised into a **feature matrix** $\mathbf{X}$ (objects as rows) and a **target vector** $\mathbf{t}$:

$$\mathbf{X} = \begin{pmatrix} - & \mathbf{x}_1 & - \\ & \vdots & \\ - & \mathbf{x}_n & - \end{pmatrix}, \quad \mathbf{t} = \begin{pmatrix} t_1 \\ \vdots \\ t_n \end{pmatrix}. \tag{2}$$

The aim of supervised learning is to exploit the training set to perform predictions on new elements $\overline{\mathbf{x}} \in \mathcal{X}$, either by directly using the training set for each prediction or, more generally, by deriving from it a **predictor**, or a set of predictors, used for the prediction.

1.5 Relations between items and their targets

Both the objects you observe and their labels come from some underlying stochastic process. The **input distribution** p_M describes the sampling likelihood of objects: $p_M(\mathbf{x})$ is the probability $\mathbf{x}$ appears in your dataset. For the target, the simplest assumption is a deterministic **ground truth function** $f : \mathcal{X} \to \mathcal{Y}$ such that $t = f(\mathbf{x})$ exactly: the features carry all the information needed, and noise is absent.

More generally, target values may depend on unobserved variables, or observations may be noisy. The relation between input and target then takes a probabilistic form, captured by a **conditional label distribution** $p_C(\cdot|\mathbf{x})$: the same object may legitimately receive different labels. Together with p_M, this induces the joint distribution $p_J(\mathbf{x}, t) = p_M(\mathbf{x})\, p_C(t|\mathbf{x})$ over object-label pairs. The functional setting is the special case where $p_C(\cdot|\mathbf{x})$ places all its mass on $f(\mathbf{x})$.

1.6 Functional predictors

In the most natural definition, a **functional predictor** is a map $h : \mathcal{X} \to \mathcal{Y}$ that assigns a predicted value $\hat{y} = h(\mathbf{x})$ to every input.

$$\mathbf{x} \xrightarrow{h} \hat{y} \in \mathcal{Y}. \tag{3}$$

For reasons that will be explained, however, it is in most cases defined as a map $h : \mathcal{X} \to \mathbb{R}$ assigning a real-valued **score** $y = h(\mathbf{x})$ to every input, with the final prediction $\hat{y} \in \mathcal{Y}$ obtained

through a task-dependent **decision rule** $d : \mathbb{R} \to \mathcal{Y}$:

$$\mathbf{x} \xrightarrow{h} y \xrightarrow{d} \hat{y} \in \mathcal{Y}. \tag{4}$$

There are three different strategies for constructing a prediction function.

1.6.1 Direct Prediction Computation

In the simplest approach, a predefined algorithm computes $y = h(\mathbf{x}, \mathcal{T})$ at prediction time using both the new item and the entire training set. No explicit learning phase exists: training examples are stored and referenced during prediction.

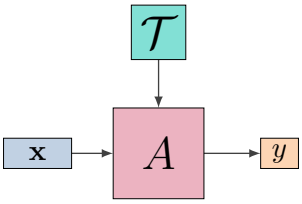


Figure 1: Direct prediction: the algorithm A takes both the new input $\mathbf{x}$ and the entire training set $\mathcal{T}$ at prediction time, producing output y without a separate learning phase.

A classic example is the *k*-**nearest neighbours** (k-NN) algorithm: to classify a new item $\mathbf{x}$, find the k training objects closest to $\mathbf{x}$ under some distance measure and output their majority class. The approach is simple, adapts naturally to different regions of the input space, and introduces no explicit model of the data.

1.6.2 Model-Based Learning Approach

The second, more typical approach defines a **hypothesis class** $\mathcal{H}$, a predefined family of functions mapping $\mathcal{X}$ to $\mathcal{Y}$, and uses a learning algorithm $\mathcal{A}$ to select the function $h_{\mathcal{T}} \in \mathcal{H}$ best approximating the training examples. Prediction then simply applies $h_{\mathcal{T}}(\mathbf{x})$.

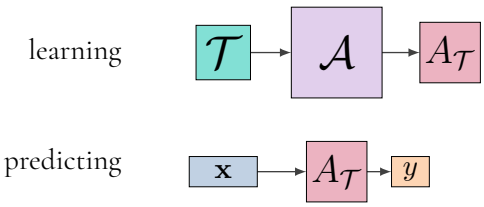


Figure 2: Model-based learning. During training (top), algorithm $\mathcal{A}$ processes $\mathcal{T}$ and produces a trained predictor $A_{\mathcal{T}}$. During prediction (bottom), $A_{\mathcal{T}}$ maps a new input $\mathbf{x}$ to an output y efficiently, without revisiting the training data.

Often $\mathcal{H}$ consists of **parametric functions**, sharing the same structure but differing in parameter values: we write $\mathcal{H}_{\boldsymbol{\theta}}$ with $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_m)$, and learning reduces to finding the parameter values best fitting the training data. In linear regression, for instance, the target is a weighted sum of features:

$$y = \sum_{i=1}^{d} w_i x_i + w_0 = \mathbf{w}^T \overline{\mathbf{x}} \tag{5}$$

where $\mathbf{w} = (w_0, w_1, \ldots, w_d)^T$ is the parameter vector and $\overline{\mathbf{x}} = (1, x_1, \ldots, x_d)^T$ includes a constant for the bias term; learning finds the weights minimizing prediction error on the training set.

1.6.3 Ensemble Learning Approach

The third approach constructs multiple predictors and combines their predictions; this **ensemble** strategy often outperforms any individual predictor, since different models capture different aspects of the data. First, s predictors $A_{\mathcal{T}}^{(1)}, \ldots, A_{\mathcal{T}}^{(s)}$ are derived from the training set; second, each is assigned a weight $w^{(i)}$ reflecting its estimated quality; third, at prediction time their outputs $y^{(i)} = h_{\mathcal{T}}^{(i)}(\mathbf{x})$ are combined using the weights.

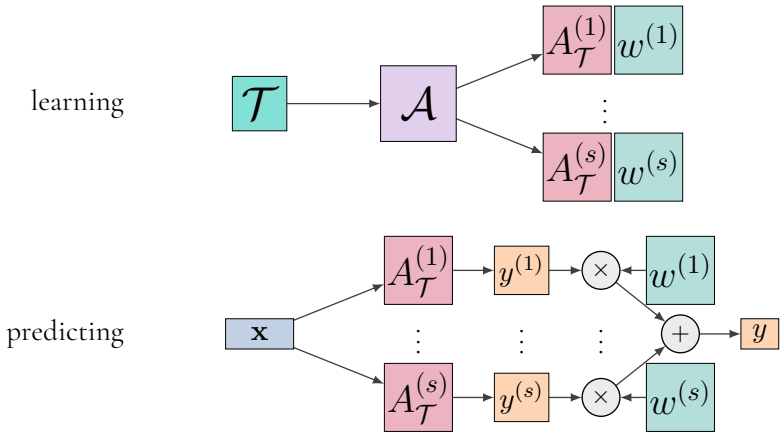


Figure 3: Ensemble learning. Training (top) produces s independent predictors $A_{\mathcal{T}}^{(1)}, \ldots, A_{\mathcal{T}}^{(s)}$ with associated weights $w^{(1)}, \ldots, w^{(s)}$. At prediction time (bottom), each predictor votes and the weighted combination yields the final output y.

For regression, we compute a weighted average:

$$h(\mathbf{x}) = \sum_{i=1}^{s} w^{(i)} h^{(i)}(\mathbf{x}) \tag{6}$$

where the weights satisfy $\sum_{i=1}^{s} w^{(i)} = 1$ and $w^{(i)} \geq 0$. For classification, we use **weighted voting**, selecting the class receiving the most weight:

$$h(\mathbf{x}) = \operatorname*{argmax}_{y \in \mathcal{Y}} \sum_{i=1}^{s} w^{(i)} \mathbb{1}[h^{(i)}(\mathbf{x}) = y]. \tag{7}$$

Direct methods are flexible and locally adaptive, model-based methods efficient and interpretable, ensemble methods robust and often more accurate.

1.7 Probabilistic predictors

A **probabilistic predictor** is a map

$$h : \mathcal{X} \to \mathcal{D}(\mathcal{Y}), \tag{8}$$

where $\mathcal{D}(\mathcal{Y})$ denotes a set of probability distributions over $\mathcal{Y}$: for each input $\mathbf{x}$, the predictor returns a distribution $h(\mathbf{x}) = p_P(\cdot|\mathbf{x})$ on $\mathcal{Y}$, rather than a single score.

A functional predictor is the special case in which all the probability mass sits on a single value. A hard prediction is obtained by applying a **decision rule** $d : \mathcal{D}(\mathcal{Y}) \to \mathcal{Y}$ directly to the distribution p_P,

$$\mathbf{x} \xrightarrow{h} p_P \xrightarrow{d} \hat{y} \in \mathcal{Y}, \tag{9}$$

exactly parallel to the functional case, where d maps $\mathbb{R} \to \mathcal{Y}$; no intermediate real-valued summary of p_P is needed.

Regression. $\mathcal{Y} = \mathbb{R}$, and the natural decision rule is the posterior mean

$$\hat{y} = d(p_P) = \mathop{\mathbb{E}}_{t \sim p_P(\cdot|\mathbf{x})}[t]\,. \tag{10}$$

Binary classification. $\mathcal{Y} = \{-1, 1\}$, and the natural decision rule is the posterior mode,

$$\hat{y} = d(p_P) = \mathop{\operatorname{argmax}}_{t \in \mathcal{Y}} p_P(t|\mathbf{x}), \tag{11}$$

Multiclass classification. $\mathcal{Y} = \{y_1, \ldots, y_K\}$, and the natural decision rule is again the posterior mode,

$$\hat{y} = d(p_P) = \mathop{\operatorname{argmax}}_{t \in \mathcal{Y}} p_P(t|\mathbf{x}). \tag{12}$$

The distribution p_P can be returned explicitly (the values $p_P(y_k)$ for a discrete $\mathcal{Y}$), through the parameters of a parametric family (e.g. mean and variance of a Gaussian), or as a method computing $p_P(t)$ for any t. The three types of predictors listed above exist also in the probabilistic case.

1.7.1 Direct prediction.

An algorithm takes $\mathbf{x}$, t, and the entire training set $\mathcal{T}$ at prediction time and returns $p_P(t|\mathbf{x})$ directly, without a separate learning phase.

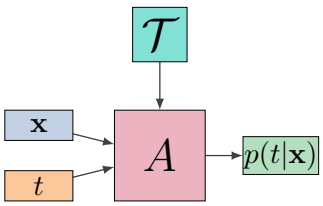


Figure 4: Direct prediction: algorithm A takes the new input $\mathbf{x}$, a target value t, and the entire training set $\mathcal{T}$ at prediction time, producing output $p(t|\mathbf{x})$ without a separate learning phase.

An example is the use of density estimation methods such as Parzen windows in classification, to compute the inverse probabilities $p(\mathbf{x}|y_i)$, from which $p_C(t \mid \mathbf{x})$ follows by Bayes' rule (given a prior distribution $p(t)$).

1.7.2 Model-based learning.

Here the role of the hypothesis class $\mathcal{H}$ is taken by a parametric distribution $\mathcal{D}$, a predefined family of conditional distributions $p(\cdot|\cdot;\boldsymbol{\theta})$. Given a training set $\mathcal{T}$, the learning algorithm $\mathcal{A}$ selects the distribution $p^*(\cdot|\cdot;\mathcal{T})$ best approximating the training examples, by a criterion discussed later. At prediction time, for an item $\mathbf{x}$, the returned distribution is $p_P = p^*(\cdot|\mathbf{x};\mathcal{T}) \in \mathcal{D}$.

1.7.3 Ensemble learning.

The fully Bayesian approach, introduced later, works as an ensemble learning approach: rather than selecting a single predictor, you derive $p(\cdot|\mathbf{x})$ by averaging predictions over a family of parametric distributions.

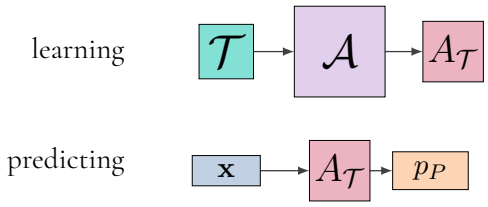


Figure 5: Model-based learning. During training (top), algorithm $\mathcal{A}$ processes $\mathcal{T}$ and produces a trained predictor $A_{\mathcal{T}}$. During prediction (bottom), $A_{\mathcal{T}}$ maps a new input $\mathbf{x}$ to a distribution p_P without revisiting the training data.

2 Measuring the quality of a predictor: the loss function

Learning means selecting a predictor within some class, which requires measuring how good a prediction is: the **risk**, the error a prediction incurs on a randomly chosen item. It is built on a **loss function** $L(h(\mathbf{x}), t)$, measuring how far the prediction for $\mathbf{x}$ is from its true output t; when $h(\mathbf{x})$ is a distribution, the comparison function is called a **scoring rule**, though "loss function" is often used loosely for both.

2.1 Loss function definition, per-element risk, and expected risk

The loss function L compares a prediction y – a score in $\mathbb{R}$, a target value in $\mathcal{Y}$ or a distribution on $\mathcal{Y}$ – with the target value $t \in \mathcal{Y}$, returning the cost of predicting y when the target is t[1]. The **per-element risk** $\mathcal{R} : \mathcal{X} \mapsto \mathbb{R}$ is the error we expect the predictor to incur on a single item $\mathbf{x}$: under a functional relation f it is simply $\mathcal{R}(\mathbf{x}) = L(y, f(\mathbf{x}))$; under a probabilistic relation it is the expectation of $L(y, t)$ over $t \sim p_C(\cdot|\mathbf{x})$.

The **expected risk** is the expectation of $\mathcal{R}(\mathbf{x})$ for $\mathbf{x} \sim p_M(\mathbf{x})$: the average error of the predictor on a randomly chosen item, our main theoretical indicator of model quality, to be minimized. Computing it, however, requires $p_J(\mathbf{x}, t) = p_C(t|\mathbf{x})p_M(\mathbf{x})$, which is unavailable by assumption: a related indicator, the **empirical risk**, is used in practice.

Four cases arise, depending on the relation between item and target (functional vs. probabilistic) and on the type of predictor (functional vs. probabilistic), governed by a single principle:

- The **type of predictor** determines the structure of the loss. A functional predictor produces a value or score $y = h(\mathbf{x})$, compared with t by a loss function; a probabilistic predictor outputs a distribution $p_P = h(\mathbf{x})$, compared with t by a **scoring rule**.
- The **relation model** determines the per-element risk. Under a deterministic relation, the risk at $\mathbf{x}$ is the loss evaluated at $t = f(\mathbf{x})$; under a probabilistic one, it is an expectation over $p_C(\cdot|\mathbf{x})$.

Table 1 summarizes the four cases.

2.2 The Empirical Risk as an Estimator of the Expected Risk

The expected risk depends on L, on p_M and on p_C (or f). L is ours to choose and p_M can usually be assumed known, but f and p_C are unknown by assumption, otherwise the task would be trivial. We do have the **empirical risk**, the average loss over the training set, viewed as a sample drawn from p_J:

$$\overline{\mathcal{R}}_{\mathcal{T}}(h) = \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} L(h(\mathbf{x}), t). \tag{13}$$

[1] As we shall see, in classification, where $\mathcal{Y}$ is discrete, a loss function comparing pairs of discrete values lacks properties necessary for learning. Thus, functional predictors are defined as predicting real scores – that the decision rule d maps to values in $\mathcal{Y}$ – which are then referred to in the **surrogate** loss function.

Setting	Loss	Per-element risk $\mathcal{R}(\mathbf{x}; h)$	Expected risk
FF case functional predictor functional relation	$L(y,\, t)$	$L(y,\, f(\mathbf{x}))$	$\mathop{\mathbb{E}}_{\mathbf{x}\sim p_M}\left[L(y, f(\mathbf{x}))\right]$
FP case functional predictor probabilistic relation	$L(y,\, t)$	$\mathop{\mathbb{E}}_{t\sim p_C(\cdot\mid\mathbf{x})}\left[L(y,t)\right]$	$\mathop{\mathbb{E}}_{(\mathbf{x},t)\sim p_J}\left[L(y,t)\right]$
PF case probabilistic predictor functional relation	$S(p_P, t)$	$S(p_P, f(\mathbf{x}))$	$\mathop{\mathbb{E}}_{\mathbf{x}\sim p_M}\left[S(p_P, f(\mathbf{x}))\right]$
PP case probabilistic predictor probabilistic relation	$S(p_P, t)$	$\mathop{\mathbb{E}}_{t\sim p_C(\cdot\mid\mathbf{x})}\left[S(p_P, t)\right]$	$\mathop{\mathbb{E}}_{(\mathbf{x},t)\sim p_J}\left[S(p_P, t)\right]$

Table 1: The four combinations of relation model and predictor type. By definition, given a predictor h, $h(\mathbf{x}) = y$ for FF and FP, while $h(\mathbf{x}) = p_P$ for PF and PP. Moving across relation types (FF $\to$ FP and PF $\to$ PP) changes only the per-element risk, because a probabilistic relation introduces an expectation of the loss over t. Moving across predictors (Cases FF $\to$ PF and Cases FP $\to$ PP) changes the loss from a scalar comparison L to a scoring rule S, because a probabilistic predictor outputs a distribution rather than a score. The expected risk definition is always derived as the expectation of the per-element risk with respect to the distribution p_M of items (let us remind that $p_J(\mathbf{x}, t) = p_M(\mathbf{x})p_C(t|\mathbf{x})$).

This is a sensible substitute since

$$\mathop{\mathbb{E}}_{(\mathbf{x},t)\sim p_J}\left[L\big(h(\mathbf{x}), t\big)\right] = \mathop{\mathbb{E}}_{\mathbf{x}\sim p_M}\left[\mathop{\mathbb{E}}_{t\sim p_C(\cdot\mid\mathbf{x})}\left[L\big(h(\mathbf{x}), t\big)\right]\right] \tag{14}$$

and

$$\mathop{\mathbb{E}}_{t\sim p_C(\cdot\mid\mathbf{x})}\left[L\big(h(\mathbf{x}), t\big)\right] = L(h(\mathbf{x}), z) \tag{15}$$

whenever $t = z$ with probability one, that is, $p_C(t|\mathbf{x}) = 1$ for $t = z$ and $p_C(t|\mathbf{x}) = 0$ otherwise. Hence the empirical risk is the sample mean used as an estimator of the expectation defining the risk, with the same definition in all four cases (Table 2).

2.2.1 General properties of the sample mean for i.i.d. variables

We first recall general properties of the sample mean of i.i.d. random variables, which apply to the empirical risk as a particular sample mean.

Unbiasedness. Let Z be a random variable with expectation $\mathbb{E}_{Z\sim p}[Z] = \mu$, and let $Z_1, \ldots, Z_n$ be n independent draws from $p(\cdot)$. The average

$$\overline{Z} = \frac{1}{n}\sum_{i=1}^{n} Z_i \tag{16}$$

is an unbiased estimator of μ: its expectation equals μ. The same holds when $Z = g(X)$ for some fixed function g of another random variable X: averaging $g(X_1), \ldots, g(X_n)$ yields an unbiased estimator of $\mathbb{E}_X[g(X)]$.

Convergence in probability and consistency. Unbiasedness says nothing about how close $\overline{Z}$ is to μ for a particular sample. What happens as the sample grows is described by consistency. A

Setting	Loss	Prediction	Empirical risk $\overline{\mathcal{R}}_{\mathcal{T}}(h)$
FF case functional predictor functional relation	$L(y,\, t)$	$y = h(\mathbf{x})$	$\dfrac{1}{\lvert\mathcal{T}\rvert} \displaystyle\sum_{(\mathbf{x},t)\in\mathcal{T}} L(h(\mathbf{x}), t)$
FP case functional predictor probabilistic relation	$L(y,\, t)$	$y = h(\mathbf{x})$	$\dfrac{1}{\lvert\mathcal{T}\rvert} \displaystyle\sum_{(\mathbf{x},t)\in\mathcal{T}} L(h(\mathbf{x}), t)$
PF case probabilistic predictor functional relation	$S(p_P, t)$	$p_P = h(\mathbf{x})$	$\dfrac{1}{\lvert\mathcal{T}\rvert} \displaystyle\sum_{(\mathbf{x},t)\in\mathcal{T}} S(p_P, t_i)$
PP case probabilistic predictor probabilistic relation	$S(p_P, t)$	$p_P = h(\mathbf{x})$	$\dfrac{1}{\lvert\mathcal{T}\rvert} \displaystyle\sum_{(\mathbf{x},t)\in\mathcal{T}} S(p_P, t_i)$

Table 2: The empirical risk in all four combinations of relation model and predictor type. As can be seen, its structure is the same in all of them: comparing the prediction returned by h with the observed value of the target.

sequence of estimators $\hat{\mu}_n$ converges in probability to μ, written $\hat{\mu}_n \xrightarrow{p} \mu$, if for every $\varepsilon > 0$,

$$\lim_{n\to\infty} P\big(|\hat{\mu}_n - \mu| > \varepsilon\big) = 0. \tag{17}$$

By the Weak Law of Large Numbers, the average of n i.i.d. random variables with finite mean converges in probability to that mean; hence, assuming only that μ exists,

$$\overline{Z} \xrightarrow{p} \mu, \qquad \text{as } n \to \infty. \tag{18}$$

The role of variance: concentration around the mean. Consistency says nothing about how quickly $\overline{Z}$ settles around μ. Here the variance enters, through the notion of concentration: a random variable *concentrates around its mean* on a scale σ when deviations much larger than σ become increasingly unlikely. The elementary tool quantifying this is Chebyshev's inequality.

Chebyshev's inequality and the variance of a sample mean

For *any* random variable X with finite mean μ and finite variance σ^2, and any $\varepsilon > 0$, Chebyshev's inequality states

$$P\big(|X - \mu| > \varepsilon\big) \;\le\; \frac{\sigma^2}{\varepsilon^2}, \tag{19}$$

with no assumption on the shape of the distribution of X: a very general, if often loose, tool. By independence of $Z_1, \ldots, Z_n$, no covariance terms survive and

$$\operatorname{Var}(\overline{Z}) \;=\; \operatorname{Var}\left(\frac{1}{n}\sum_{i=1}^{n} Z_i\right) \;=\; \frac{1}{n^2}\sum_{i=1}^{n} \operatorname{Var}(Z_i) \;=\; \frac{\sigma^2}{n}. \tag{20}$$

Applying Chebyshev's inequality with $X = \overline{Z}$,

$$P\big(|\overline{Z} - \mu| > \varepsilon\big) \;\le\; \frac{\sigma^2}{n\varepsilon^2}, \tag{21}$$

which recovers consistency with an explicit rate: the probability of a sizeable deviation shrinks as $1/n$, scaled by σ^2. Equivalently, $\overline{Z}$ concentrates around μ on the scale $\sigma/\sqrt{n}$, its **standard error**.

2.2.2 Specializing to the empirical risk

The empirical risk is an instance of this construction, with g specialized to the loss L and X to the pair $(\mathbf{x}, t)$. Let $(\mathbf{x}_1, t_1), \ldots, (\mathbf{x}_n, t_n)$ be the pairs in $\mathcal{T}$, drawn i.i.d. from p_J, and let L be a fixed loss. Given a predictor h, define

$$\theta = \mathcal{R}_{p_J}(h) = \mathop{\mathbb{E}}_{(\mathbf{x},t)\sim p_J} \left[L(h(\mathbf{x}), t)\right], \qquad \hat{\theta} = \overline{\mathcal{R}}_{\mathcal{T}}(h) = \frac{1}{n}\sum_{i=1}^{n} L(h(\mathbf{x}_i), t_i). \tag{22}$$

The losses $L(h(\mathbf{x}_i), t_i)$ are i.i.d., and $\hat{\theta}$ is their sample mean, with θ in the role of μ. Three consequences follow.

Unbiasedness. For any loss and any p_J for which θ exists,

$$\mathop{\mathbb{E}}_{\mathcal{T}\sim p_J^n} \left[\hat{\theta}\right] = \theta, \tag{23}$$

the expectation being taken over the random draw of the training set, whatever its size.

Consistency. By the Weak Law of Large Numbers, assuming only that θ exists,

$$\overline{\mathcal{R}}_{\mathcal{T}}(h) \xrightarrow{p} \mathcal{R}_{p_J}(h), \qquad \text{as } |\mathcal{T}| \to \infty. \tag{24}$$

Rate. Writing $\sigma_L^2(h) = Var_{(\mathbf{x},t)\sim p_J}\left[L(h(\mathbf{x}), t)\right]$ for the variance of the per-example loss, the notebox above gives $\mathrm{Var}(\hat{\theta}) = \sigma_L^2(h)/n$ and hence

$$P\big(|\hat{\theta} - \theta| > \varepsilon\big) \;\le\; \frac{\sigma_L^2(h)}{n\varepsilon^2}\,: \tag{25}$$

the empirical risk of a *fixed* h concentrates around its expected risk on the scale $\sigma_L(h)/\sqrt{n}$.

Our ultimate goal, however, is performance on unseen data, expressed by the expected risk: the relation between the two is the main topic of **statistical learning theory**.

2.3 Minimization of the empirical risk and its issues

The empirical risk allows to identify the "best" predictor in a given set $\mathcal{H}$ (the **hypothesis class**) via **empirical risk minimization** (ERM),

$$h_{\mathcal{T}} \;=\; \mathop{\mathrm{argmin}}_{h\in\mathcal{H}} \overline{\mathcal{R}}_{\mathcal{T}}(h). \tag{26}$$

Two separate questions arise. The first is *statistical*: the predictor we actually want is $h^* = \mathop{\mathrm{argmin}}_{h\in\mathcal{H}} \mathcal{R}_{p_J}(h)$, the one with minimal *true* risk; does $h_{\mathcal{T}}$ approach h^* as the training set grows? The second is *computational*: $\mathcal{H}$ may be a space of functions in which optimization is hard. We take up the first question first, since its answer already tells us which $\mathcal{H}$ are admissible.

2.3.1 Actually reaching the target from a sample

Even when $f(\mathbf{x})$ — or the appropriate statistic of $p_C(\cdot \mid \mathbf{x})$ — uniquely minimizes the expected risk, does minimizing the *empirical* risk return something close to it? The law of large numbers alone does not say so.

What convergence in probability actually promises. Fix in advance one predictor h, chosen without looking at $\mathcal{T}$. As n grows, the empirical risk of *this fixed* h concentrates around its true risk:

$$P\big(|\overline{\mathcal{R}}_{\mathcal{T}}(h) - \mathcal{R}_{p_J}(h)| > \varepsilon\big) \xrightarrow{n\to\infty} 0 \qquad \text{for every fixed } \varepsilon > 0. \tag{27}$$

The quantification matters: *first* h is fixed, *then* n is sent to infinity.

Why $h_{\mathcal{T}}$ breaks the hypothesis the guarantee needs. The predictor $h_{\mathcal{T}}$ returned by ERM is not fixed in advance: it is selected *using* $\mathcal{T}$, so it is itself random, and a different training set generally returns a different $h_{\mathcal{T}}$. This is exactly the situation Equation (27) does not cover.

The critical issue: minimization and the limit do not commute. It is tempting to argue that, since the empirical risk converges to the true risk for every fixed h, minimizing one or the other must agree in the limit, giving $h_{\mathcal{T}} \to h^*$. This silently exchanges two operations that do not commute in general:

$$\lim_{n\to\infty} \operatorname*{argmin}_{h\in\mathcal{H}} \overline{\mathcal{R}}_{\mathcal{T}}(h) \quad \stackrel{?}{=} \quad \operatorname*{argmin}_{h\in\mathcal{H}} \lim_{n\to\infty} \overline{\mathcal{R}}_{\mathcal{T}}(h). \tag{28}$$

The right-hand side is h^* by definition; the left-hand side is what ERM produces for ever-larger samples. Equality holds only under an additional condition, identified in what follows.

Why the finite case is safe: a union bound. Let $\mathcal{H} = \{h_1, \dots, h_K\}$ be finite, with K fixed in advance. Equation (27) applies to each candidate, and the probability that *at least one* has a large gap is controlled by the union bound.

The union bound for a finite hypothesis class

For every fixed $\varepsilon > 0$,

$$P\Big(\exists\, k :\ |\overline{\mathcal{R}}_{\mathcal{T}}(h_k) - \mathcal{R}_{p_J}(h_k)| > \varepsilon\Big) \ \le\ \sum_{k=1}^{K} P\big(|\overline{\mathcal{R}}_{\mathcal{T}}(h_k) - \mathcal{R}_{p_J}(h_k)| > \varepsilon\big). \tag{29}$$

As $n \to \infty$, each of the K terms on the right tends to 0; since K is finite and does not grow with n, so does their sum, and

$$P\Big(\forall\, k :\ |\overline{\mathcal{R}}_{\mathcal{T}}(h_k) - \mathcal{R}_{p_J}(h_k)| \le \varepsilon\Big) \xrightarrow{n\to\infty} 1. \tag{30}$$

With probability tending to 1, *every* candidate has empirical risk close to its true risk *simultaneously*: the guarantee therefore covers $h_{\mathcal{T}}$, whichever candidate it turns out to be.

Where the argument breaks. This rests entirely on $\mathcal{H}$ being fixed and finite. If $\mathcal{H}$ is instead a large, essentially unrestricted class[2], the guarantee no longer holds, for reasons worth understanding precisely.

Definition

Denote by $p_n(h)$ the probability, with respect to the sampling of the training set, that the empirical and true risk of a *fixed* predictor h differ by more than a fixed margin ε at sample

[2] What exactly "unrestricted" means here is made precise later, via the notion of VC dimension: informally, a class in which the number of genuinely distinguishable candidates, on any n points, keeps growing with n rather than staying bounded.

size n:

$$p_n(h) \;=\; P\big(|\overline{\mathcal{R}}_{\mathcal{T}}(h) - \mathcal{R}_{p_J}(h)| > \varepsilon\big). \tag{31}$$

By Equation (25), $p_n(h) \le \sigma_L^2(h)/(n\varepsilon^2) \to 0$ for each fixed h, possibly at different speeds for different candidates; with K candidates fixed in advance, $\sum_{k=1}^{K} p_n(h_k) \to 0$ as $n \to \infty$.

From one predictor to many. Fix n and let the *number* of candidates grow: an event of small probability p for each of K independent quantities occurs for at least one of them with probability about $1-(1-p)^K \to 1$. Equivalently, with n fixed the bound $\sum_k p_n(h_k) \le K \max_k p_n(h_k)$ of Equation (29) grows with K: enough candidates make it almost certain that *some* predictor has a large gap.

This statement is two-sided: an anomalous candidate looking *worse* than it is is harmless, since ERM will not select it, while one looking *better* is exactly what ERM is liable to select. The next paragraphs settle the direction and size of the effect.

ERM as a minimum, and its systematic optimism. The direction of the bias follows from the definition of ERM alone.

The optimism of the minimum

For each h, $\mathbb{E}_{\mathcal{T}:|\mathcal{T}|=n}\big[\overline{\mathcal{R}}_{\mathcal{T}}(h)\big] = \mathcal{R}_{p_J}(h)$ (unbiasedness). Since $\overline{\mathcal{R}}_{\mathcal{T}}(h_{\mathcal{T}}) = \min_h \overline{\mathcal{R}}_{\mathcal{T}}(h) \le \overline{\mathcal{R}}_{\mathcal{T}}(h)$ for every $h \in \mathcal{H}$ and every $\mathcal{T}$, taking expectations gives

$$\mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}\big[\overline{\mathcal{R}}_{\mathcal{T}}(h_{\mathcal{T}})\big] \;\le\; \mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}\big[\overline{\mathcal{R}}_{\mathcal{T}}(h)\big] \;=\; \mathcal{R}_{p_J}(h) \tag{32}$$

for every fixed h, in particular $h = h^*$. Conversely, $\mathcal{R}_{p_J}(h^*) \le \mathcal{R}_{p_J}(h_{\mathcal{T}})$ for every $\mathcal{T}$, since h^* minimizes $\mathcal{R}_{p_J}$ over $\mathcal{H}$. Combining,

$$\mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}\big[\overline{\mathcal{R}}_{\mathcal{T}}(h_{\mathcal{T}})\big] \;\le\; \mathcal{R}_{p_J}(h^*) \;\le\; \mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}\big[\mathcal{R}_{p_J}(h_{\mathcal{T}})\big]. \tag{33}$$

On average, the empirical risk of the predictor selected by ERM *underestimates* its true risk — the familiar optimistic bias of using the minimum of several unbiased estimators. This settles the *direction* of the bias but not its *size*, and holds for any $\mathcal{H}$ (for a single candidate Equation (33) reduces to an equality). It remains to show that the gap grows with the richness of $\mathcal{H}$.

Quantifying the gap: K equally good candidates. Consider K candidates $h_1, \dots, h_K$ sharing the *same* true risk $\mathcal{R}_{p_J}(h_k) = \mu$, so that any apparent drop of $\overline{\mathcal{R}}_{\mathcal{T}}(h_{\mathcal{T}})$ below μ is purely an artefact of comparing noisy estimates of the same quantity. Assume the loss is bounded, say in $[0,1]$. Then each $\overline{\mathcal{R}}_{\mathcal{T}}(h_k)$ concentrates around μ on the scale $\sigma_n = O(1/\sqrt{n})$, and a standard bound on the minimum of K such quantities gives

$$\mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}\big[\mu \;-\; \overline{\mathcal{R}}_{\mathcal{T}}(h_{\mathcal{T}})\big] \;\le\; \sigma_n\sqrt{2\log K}. \tag{34}$$

The bound holds regardless of the relations among candidates, but can be very *loose* when they are strongly correlated: if they behave almost identically on $\mathcal{T}$, their minimum is barely below any single one. It is instead *tight* when the candidates behave independently, or are sufficiently different, on the sample:

$$\mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n}\Big[\mu - \min_k \overline{\mathcal{R}}_{\mathcal{T}}(h_k)\Big] \;=\; \sigma_n\sqrt{2\log K}\,\big(1+o(1)\big) \qquad \text{as } K \to \infty. \tag{35}$$

The optimism thus grows as $\sqrt{\log K/n}$ in the number of genuinely distinct candidates, without bound as $K \to \infty$ at fixed n. Dropping the equal-risk assumption mixes this optimism with genuine quality differences, but a gap of the same order persists.

The $K \to \infty$ limit and the interpolating function. For an essentially unrestricted $\mathcal{H}$ the phenomenon becomes deterministic. Let $\mathcal{Y} = \mathbb{R}$, the loss be quadratic, $\mathcal{H}$ be *all* measurable functions $\mathcal{X} \to \mathbb{R}$, and $t = f(\mathbf{x}) + \varepsilon$ with genuine noise ε. For any sample of size n, $\mathcal{H}$ contains a function $h_{\mathcal{T}}$ interpolating every training pair, $h_{\mathcal{T}}(\mathbf{x}_i) = t_i$, and behaving arbitrarily elsewhere — a Runge-like interpolant, or simply a lookup table returning t_i on $\mathbf{x}_i$ and, say, 0 elsewhere. This $h_{\mathcal{T}}$ has empirical risk *exactly zero* for every $\mathcal{T}$ and every n, with certainty rather than with high probability, while its true risk can be made as large as desired (choosing f far from 0, or by the amplitude of the oscillations between data points). The gap $\mathcal{R}_{p_J}(h_{\mathcal{T}}) - \mathcal{R}_{p_J}(h^*)$ is thus *unbounded* at every n: the deterministic endpoint of the phenomenon quantified above for finite K.

A sharper formalization: one predictor per sample size. Both phenomena are captured by the worst-case excess risk at sample size n,

$$\Delta_n \;:=\; \sup_{\mathcal{T}\,:\,|\mathcal{T}|=n} \Big[\mathcal{R}_{p_J}(h_{\mathcal{T}}) - \mathcal{R}_{p_J}(h^*)\Big], \tag{36}$$

the largest true-risk gap of ERM's output, over all training sets of size n; let h_n denote a predictor realizing it (up to an arbitrarily small slack). ERM's output approaches h^* exactly when $\Delta_n \to 0$, which the interpolating construction shows can fail. What controls Δ_n in general?

Relating Δ_n to uniform convergence.

Bounding the excess risk

Fix any training set $\mathcal{T}$ of size n and add and subtract the empirical risks of $h_{\mathcal{T}}$ and h^*:

$$\mathcal{R}_{p_J}(h_{\mathcal{T}}) - \mathcal{R}_{p_J}(h^*) = \underbrace{\Big[\mathcal{R}_{p_J}(h_{\mathcal{T}}) - \overline{\mathcal{R}}_{\mathcal{T}}(h_{\mathcal{T}})\Big]}_{\text{(a)}} + \underbrace{\Big[\overline{\mathcal{R}}_{\mathcal{T}}(h_{\mathcal{T}}) - \overline{\mathcal{R}}_{\mathcal{T}}(h^*)\Big]}_{\text{(b)}} + \underbrace{\Big[\overline{\mathcal{R}}_{\mathcal{T}}(h^*) - \mathcal{R}_{p_J}(h^*)\Big]}_{\text{(c)}}.$$

Term (b) is never positive, since $h_{\mathcal{T}}$ minimizes the empirical risk over $\mathcal{H} \ni h^*$ (the only place where $h_{\mathcal{T}}$ being an ERM output is used). Terms (a) and (c) are bounded in absolute value by $\sup_{h\in\mathcal{H}} |\overline{\mathcal{R}}_{\mathcal{T}}(h) - \mathcal{R}_{p_J}(h)|$. Therefore

$$\mathcal{R}_{p_J}(h_{\mathcal{T}}) - \mathcal{R}_{p_J}(h^*) \;\le\; 2 \sup_{h\in\mathcal{H}} \big|\overline{\mathcal{R}}_{\mathcal{T}}(h) - \mathcal{R}_{p_J}(h)\big|, \tag{37}$$

and, taking the supremum over all $\mathcal{T}$ of size n,

$$\Delta_n \;\le\; 2 \sup_{\mathcal{T}:|\mathcal{T}|=n} \sup_{h\in\mathcal{H}} \big|\overline{\mathcal{R}}_{\mathcal{T}}(h) - \mathcal{R}_{p_J}(h)\big|. \tag{38}$$

So $\Delta_n \to 0$ whenever empirical risk converges to true risk *uniformly* across $\mathcal{H}$, rather than separately for each candidate: the extension of the finite-class union bound to the entire class.

The remedy. What repairs every failure traced in this section is the uniform convergence of Equation (38),

$$\sup_{h\in\mathcal{H}}\left|\overline{\mathcal{R}}_{\mathcal{T}}(h)-\mathcal{R}_{p_J}(h)\right| \xrightarrow{n\to\infty} 0, \tag{39}$$

governed by the complexity of $\mathcal{H}$ — its VC dimension, Rademacher complexity, or metric entropy. Whenever it holds, $\Delta_n \to 0$ and the per-n ERM outputs, including the worst-case (h_n), converge in risk to h^*: a genuine theorem, not an automatic consequence of the law of large numbers, obtained by ruling out classes rich enough to supply a fresh overfitting witness at every sample size. *This is the first, statistical reason $\mathcal{H}$ cannot be left unrestricted.*

2.3.2 Restricting $\mathcal{H}$ for computational reasons too

A second, independent reason points in the same direction: searching a space of functions for the empirical risk minimizer is not directly amenable to standard optimization tools (gradients, line searches), which are built for spaces of numbers.

The standard remedy is to restrict $\mathcal{H}$ to a **parametric family**: fix a functional form $h_{\boldsymbol{\theta}}$ and let $\boldsymbol{\theta} = (\theta_0, \theta_1, \ldots, \theta_d)$ range over $\mathbb{R}^{d+1}$. Each $\boldsymbol{\theta}$ picks out one function in $\mathcal{H}$, the search over functions becomes a search over parameter vectors, and training becomes finding the $\boldsymbol{\theta}$ minimizing the empirical risk. This restriction is also exactly the kind of complexity control required above: a family indexed by $d+1$ parameters typically has complexity growing only with d, so a modest d makes both gradient-based search and uniform convergence (hence $\Delta_n \to 0$) possible.

The empirical risk then becomes an ordinary function of $d+1$ real variables, to which gradient-based optimization applies, whether $h_{\boldsymbol{\theta}}$ is a linear model, a fixed-depth tree or a neural network; the number and structure of the parameters also control the capacity of $\mathcal{H}$.

For the parametric case that occupies most of these notes, we reformulate empirical risk minimization as minimizing the overall loss:

$$\mathcal{L}(\boldsymbol{\theta};\mathcal{T}) = \sum_{i=1}^{n} L_i(\boldsymbol{\theta}), \tag{40}$$

where we sum the individual loss terms $L_i = L(\boldsymbol{\theta};\mathbf{x}_i,t_i)$ over each data point $(\mathbf{x}_i,t_i)$ in the training set, making explicit the dependence on both the parameters and the training example.

2.3.3 Comparing predictors on an independent test set

Every failure traced in Section 2.3 has one root cause: the empirical risk was asked to serve *two* roles at once — selecting $h_{\mathcal{T}}$ and certifying its quality — on the same sample. The machinery above (union bound, optimism bound of Equation (34), uniform convergence, VC dimension) repairs this *theoretically*, through a guarantee holding for every candidate in $\mathcal{H}$ at once. There is a far cheaper, purely empirical fix.

Why an independent test set restores the guarantee. Equation (27) requires a predictor fixed in advance, a hypothesis $h_{\mathcal{T}}$ violates. Suppose instead that $h_{\mathcal{T}}$ is evaluated on a second, independent **test set** $\mathcal{T}'$, drawn i.i.d. from p_J, disjoint from $\mathcal{T}$ and never consulted during selection. Conditionally on $\mathcal{T}$, $h_{\mathcal{T}}$ is simply *some fixed function* with respect to $\mathcal{T}'$, and the hypothesis of Equation (27) is restored by construction:

$$P\left(\left|\overline{\mathcal{R}}_{\mathcal{T}'}(h_{\mathcal{T}})-\mathcal{R}_{p_J}(h_{\mathcal{T}})\right|>\varepsilon \;\middle|\; \mathcal{T}\right) \xrightarrow{|\mathcal{T}'|\to\infty} 0, \tag{41}$$

with no supremum over $\mathcal{H}$ and no complexity measure involved. Consequently $\mathcal{H}$ can be as rich as convenient during training — even the interpolating construction above — since $\overline{\mathcal{R}}_{\mathcal{T}'}(h_{\mathcal{T}})$ reports the truth regardless. The gap $\overline{\mathcal{R}}_{\mathcal{T}'}(h_{\mathcal{T}}) - \overline{\mathcal{R}}_{\mathcal{T}}(h_{\mathcal{T}})$ is the practical substitute for the quantity $\sup_{h\in\mathcal{H}}|\overline{\mathcal{R}}_{\mathcal{T}}(h)-\mathcal{R}_{p_J}(h)|$ of Equation (38): generalization is measured after the fact rather than proved in advance.

The limit: comparing several predictors reproduces ERM. The fix holds only for a *single*, already-fixed $h_{\mathcal{T}}$, evaluated *once*. If $\mathcal{T}'$ is used to *choose* among K candidates $h_{\mathcal{T}}^{(1)}, \ldots, h_{\mathcal{T}}^{(K)}$ (different algorithms, architectures or hyperparameters), this is empirical risk minimization over those K candidates, with $\mathcal{T}'$ in the role of $\mathcal{T}$, and every result above transfers verbatim. Comparing a small K, fixed in advance, once, is the safe union-bound regime; repeatedly inspecting performance on $\mathcal{T}'$ and adjusting the model reproduces the optimism of Equation (33), at the rate of Equation (34): the reported $\overline{\mathcal{R}}_{\mathcal{T}'}$ of the chosen configuration underestimates its true risk by an amount growing as $\sqrt{\log K/|\mathcal{T}'|}$. A test set used for model selection has thus become a second *training* set, and reporting its risk as an honest estimate repeats the same error one level removed.

The practical resolution. Hence the standard three-way split: a *training* set, on which $\mathcal{H}$ can be as rich as convenient; a *validation* set for model and hyperparameter selection, safe only as long as the number of compared configurations stays small and fixed; and a *test* set used *exactly once*, on the final model — the one point where Equation (27) applies without qualification, giving an unbiased estimate of the true risk.

2.4 Bayes Predictors and Bayes Risk

The expected risk cannot be minimized in practice, since it requires knowing p_M and $p_{C'}$. It is nonetheless worth asking: if this knowledge *were* available, what would the best possible predictor look like? The answer gives no algorithm, but an ideal reference point against which any data-driven predictor can be measured: a predictor knowing exactly, for every $\mathbf{x}$, how likely each target value t is under $p_C(\cdot|\mathbf{x})$.

Formally, the Bayes predictor h^B minimizes the expected risk over the class of *all* measurable functions[3] $h : \mathcal{X} \to \mathbb{R}$, with no restriction on its form:

$$h^B = \operatorname*{argmin}_h \mathcal{R}_{p_J}(h). \tag{42}$$

This unrestricted search makes h^B an absolute, algorithm-independent benchmark. The minimal value $\mathcal{R}^B_{p_J} = \mathcal{R}_{p_J}(h^B)$ is the Bayes risk, the error left even for the best conceivable predictor, arising whenever $p_C(\cdot|\mathbf{x})$ spreads mass over more than one value of t. If two patients with identical features sometimes recover and sometimes do not, no predictor can foresee the individual outcome, only report the correct probability of recovery: the Bayes risk is *irreducible*, reflecting noise intrinsic to the phenomenon rather than ignorance that more data could dispel.

Two remarks. First, h^B depends on the loss L, not only on p_J: different losses extract different statistics from the same posterior $p_C(t|\mathbf{x})$ — squared loss yields the conditional mean, absolute loss the conditional median, and so on.

Example

Suppose, for fixed $\mathbf{x}$, that $t = 0$ with probability 0.9 and $t = 10$ with probability 0.1. Under squared loss the Bayes-optimal prediction is the conditional mean, $0.9 \cdot 0 + 0.1 \cdot 10 = 1$, a value t never takes; under absolute loss it is the conditional median, 0. Same posterior, different losses, different optimal predictions.

[3] A measurable set is a subset of $\mathcal{X}$ belonging to the collection of subsets over which probabilities are defined: on spaces such as $\mathbb{R}^d$ one cannot consistently assign a probability to *every* subset, so probabilities are restricted to a rich collection of "well-behaved" sets, containing intervals and anything built from them. A function $h : \mathcal{X} \to \mathbb{R}$ is measurable if, for every interval $(a, b) \subset \mathbb{R}$, the set $\{\mathbf{x} \in \mathcal{X} : h(\mathbf{x}) \in (a, b)\}$ is measurable, which guarantees that expectations such as $\mathbb{E}_{\mathbf{x}\sim p_M}[L(h(\mathbf{x}), f(\mathbf{x}))]$ are well defined. This is a very mild regularity condition, imposing no constraint on the shape or complexity of h: every function one would realistically compute is measurable, so "all measurable functions" essentially means "all functions".

Second, computing h^B requires $p_C(\cdot|\mathbf{x})$ (or $f(\mathbf{x})$), exactly what learning assumes to be unavailable: the Bayes predictor is a benchmark, never a practical output of training.

Restricting the search to $\mathcal{H}$ generally worsens the achievable risk, since $\mathcal{H}$ may not contain h^B: no linear function reproduces a genuinely nonlinear relation. Let $h^*_{\mathcal{H}} = \operatorname{argmin}_{h\in\mathcal{H}} \mathcal{R}_{p_J}(h)$ be the best predictor in $\mathcal{H}$, still computed under perfect knowledge of p_J, and $h_{\mathcal{T}} \in \mathcal{H}$ the predictor actually returned by learning on the finite sample $\mathcal{T}$: the only one of the three we can compute, and generally the worst. Inserting $h^*_{\mathcal{H}}$ as an intermediate reference point decomposes the excess risk of $h_{\mathcal{T}}$ over the Bayes risk:

$$\mathcal{R}_{p_J}(h_{\mathcal{T}}) - \mathcal{R}^B_{p_J} = \underbrace{\left[\mathcal{R}_{p_J}(h_{\mathcal{T}}) - \mathcal{R}_{p_J}(h^*_{\mathcal{H}})\right]}_{\text{estimation error}} + \underbrace{\left[\mathcal{R}_{p_J}(h^*_{\mathcal{H}}) - \mathcal{R}^B_{p_J}\right]}_{\text{approximation error}}. \tag{43}$$

Both terms are nonnegative, since $h^*_{\mathcal{H}}$ and h^B are risk minimizers over their respective search spaces: the total gap splits into two separate, additive sources of suboptimality.

The **approximation error**, $\mathcal{R}_{p_J}(h^*_{\mathcal{H}}) - \mathcal{R}^B_{p_J}$, measures the risk given up by searching within $\mathcal{H}$ rather than over all functions. It compares two idealized predictors, so it depends only on how well $\mathcal{H}$ matches the true relation between $\mathbf{x}$ and t, not on the training set or the algorithm; enlarging $\mathcal{H}$ can only reduce it. It is associated with **underfitting**: a restrictive class fails to represent the true relation however carefully it is fit, even with infinite data.

The **estimation error**, $\mathcal{R}_{p_J}(h_{\mathcal{T}}) - \mathcal{R}_{p_J}(h^*_{\mathcal{H}})$, compares two predictors in the same $\mathcal{H}$, differing only in the information used to select them: exact knowledge of p_J versus a finite, noisy sample. It shrinks as $|\mathcal{T}| \to \infty$, but grows with the flexibility of $\mathcal{H}$, since richer classes are harder to estimate reliably: a learning algorithm may latch onto a function fitting the idiosyncrasies of $\mathcal{T}$ rather than the underlying pattern. It is the term associated with **overfitting**.

For instance, with a smooth quadratic relation plus noise, linear functions have large approximation error but, having few parameters, small estimation error; degree-ten polynomials have negligible approximation error but, with few examples, tend to wildly oscillating fits far from $h^*_{\mathcal{H}}$, i.e. large estimation error. Enlarging $\mathcal{H}$ shrinks the first term while inflating the second: a tension closely related, though not identical, to the **bias-variance tradeoff** discussed in Section 3.

What remains once both controllable errors vanish is the Bayes risk, which no choice of $\mathcal{H}$, amount of data or algorithm can remove. Recognizing which of the two controllable terms dominates — underfitting or overfitting — is a central diagnostic question of supervised learning.

3 The bias-variance decomposition

The decomposition of Equation (43) concerns the predictor $h_{\mathcal{T}}$ learned from one specific training set. A complementary view analyses the learning procedure itself: since $h_{\mathcal{T}}$ is random, its error can be studied *on average over training sets*, pointwise at each $\mathbf{x}$, separating what the procedure systematically predicts (**bias**) from its sensitivity to the particular sample (**variance**).

3.1 The general decomposition

Fix an item $\mathbf{x}$ and consider the predictions $h_{\mathcal{T}}(\mathbf{x})$ as $\mathcal{T}$ varies over training sets of size n. Their "center" is summarized by the **main prediction**

$$\bar{h}(\mathbf{x}) = \operatorname*{argmin}_z \mathop{\mathbb{E}}_{\mathcal{T}:|\mathcal{T}|=n} \left[L\big(h_{\mathcal{T}}(\mathbf{x}),\, z\big)\right], \tag{44}$$

the value which, taken as a target, is on average closest to the predictions under L: an idealized quantity, defined at fixed n.

Functional relation. If $t = f(\mathbf{x})$, then $h^B(\mathbf{x}) = f(\mathbf{x})$ for any loss with $L(z,z) = 0 < L(y,z)$ when $y \neq z$, and adding and subtracting $L(\bar{h}(\mathbf{x}), f(\mathbf{x}))$ gives

$$\mathop{\mathbb{E}}_{\mathcal{T}}\left[L\big(h_{\mathcal{T}}(\mathbf{x}), f(\mathbf{x})\big)\right] \;=\; \underbrace{L\big(\bar{h}(\mathbf{x}), f(\mathbf{x})\big)}_{\text{bias}} \;+\; \underbrace{\mathop{\mathbb{E}}_{\mathcal{T}}\left[L\big(h_{\mathcal{T}}(\mathbf{x}), f(\mathbf{x})\big) - L\big(\bar{h}(\mathbf{x}), f(\mathbf{x})\big)\right]}_{\text{variance}}. \tag{45}$$

Probabilistic relation: including the irreducible risk. If $t \sim p_C(\cdot|\mathbf{x})$, even h^B incurs a nonzero risk. Adding and subtracting the loss of both $h^B(\mathbf{x})$ and $\bar{h}(\mathbf{x})$ inside the expectation over t yields

$$\begin{aligned}\mathop{\mathbb{E}}_{\mathcal{T}}\left[\mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})}\left[L\big(h_{\mathcal{T}}(\mathbf{x}), t\big)\right]\right] &= \underbrace{\mathop{\mathbb{E}}_{t}\left[L\big(h^B(\mathbf{x}), t\big)\right]}_{\text{irreducible risk } C(\mathbf{x})} + \underbrace{\mathop{\mathbb{E}}_{t}\left[L\big(\bar{h}(\mathbf{x}), t\big) - L\big(h^B(\mathbf{x}), t\big)\right]}_{\text{bias } B(\mathbf{x})} \\ &\quad + \underbrace{\mathop{\mathbb{E}}_{\mathcal{T}}\left[\mathop{\mathbb{E}}_{t}\left[L\big(h_{\mathcal{T}}(\mathbf{x}), t\big) - L\big(\bar{h}(\mathbf{x}), t\big)\right]\right]}_{\text{variance } V(\mathbf{x})}.\end{aligned}$$

$C(\mathbf{x})$ is the pointwise Bayes risk of Section 2.4, fixed by p_C and L alone; Equation (45) is recovered when $p_C(\cdot|\mathbf{x})$ is a point mass at $f(\mathbf{x})$, which makes $C(\mathbf{x}) = 0$.

The identity holds for *every* loss and every choice of $\bar{h}$, since the terms added and subtracted cancel out: by itself, it says nothing about the problem. Its content lies entirely in what the three terms turn out to be.

3.2 Interpretability, and relation with the error decomposition

When are bias and variance meaningful? For the names "bias" and "variance" to be justified, the terms of the decomposition should have the following properties:

(i) $B(\mathbf{x}) \geq 0$, with $B(\mathbf{x}) = 0$ if and only if $\bar{h}(\mathbf{x}) = h^B(\mathbf{x})$: the bias depends only on where the main prediction sits;

(ii) $V(\mathbf{x}) \geq 0$, and $V(\mathbf{x})$ depends only on the spread of the predictions $h_{\mathcal{T}}(\mathbf{x})$ around $\bar{h}(\mathbf{x})$, not on $p_C(\cdot|\mathbf{x})$;

(iii) the terms combine additively, so that reducing one, with the others fixed, reduces the total.

Property (i) holds for any loss, by definition of h^B. Properties (ii) and (iii) do not: for a general loss the variance term, being a difference of expected losses against the random target, mixes the fluctuation of the predictions with the noise of the target, and can even be negative.

Bregman divergences. There is a broad class of losses for which all three properties hold. Informally, a **Bregman divergence** measures the cost of predicting y when the truth is t by taking a strictly convex function ϕ and asking how badly its tangent line at y approximates ϕ at t:

$$D_\phi(t,y) \;=\; \phi(t) - \phi(y) - \phi'(y)\,(t-y) \;\geq\; 0. \tag{46}$$

By convexity this gap is never negative, and it vanishes only when $y = t$. Different choices of ϕ give different losses: $\phi(y) = y^2$ gives the quadratic loss $(t-y)^2$. What makes these losses special is that their expected loss always splits into a part due to the noise and a part depending only on how far the prediction is from the conditional mean $\mu(\mathbf{x})$:

$$\mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})}\left[D_\phi(t,y)\right] \;=\; \mathop{\mathbb{E}}_{t\sim p_C(\cdot|\mathbf{x})}\left[D_\phi(t,\mu(\mathbf{x}))\right] \;+\; D_\phi\big(\mu(\mathbf{x}), y\big). \tag{47}$$

The cross term that would mix the two cancels because $\mathbb{E}_t\left[t - \mu(\mathbf{x})\right] = 0$. Two consequences follow:

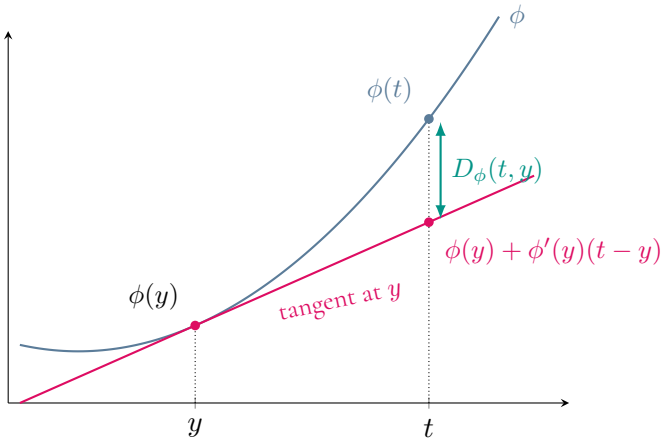


Figure 6: Bregman divergence: the gap at t between the convex function ϕ and its tangent line at y. By convexity the tangent never lies above ϕ, so the gap is non-negative, and it vanishes only when $y = t$.

- The Bayes predictor is the conditional mean, $h^B(\mathbf{x}) = \mu(\mathbf{x})$, whatever ϕ is. Conversely, under mild regularity conditions, Bregman divergences are essentially the only losses with this property.
- The same split applies to the predictions themselves. Treating $h_{\mathcal{T}}(\mathbf{x})$ as random around a suitable center, the expected loss separates into the cost of that center plus a spread around it. For this to work, the main prediction $\bar{h}(\mathbf{x})$ must be the average of the predictions taken after transforming them by ϕ', rather than their ordinary average.

Losses with an interpretable decomposition. Several losses widely used in practice are Bregman divergences. Of specific interest in our framework are:

- the quadratic loss, $\phi(y) = y^2$. Here $\bar{h}(\mathbf{x})$ is the ordinary mean of the predictions, and the decomposition is the classical noise + bias^2 + variance;
- the log loss (cross-entropy) for probabilistic predictors, generated by the negative entropy $\phi(p) = \sum_k p_k \log p_k$, for which D_ϕ is the Kullback–Leibler divergence.

Many other common losses are not Bregman divergences: the absolute loss, the Huber loss, the hinge loss and the 0–1 loss. For these, the decomposition still holds as an identity, but its variance term may depend on the noise of the target, and may even be negative.

Relation with the approximation/estimation decomposition. The two decompositions look at the same error from different angles. Approximation and estimation error refer to the predictor $h_{\mathcal{T}}$ learned from one training set, and compare it with $h^*_{\mathcal{H}}$ and h^B; bias and variance describe the learning procedure on average over training sets, and use $\bar{h}$ as reference. For any loss, averaging the bias-variance decomposition over $\mathbf{x} \sim p_M$ (the irreducible term becomes $\mathcal{R}^B_{p_J}$) and the error decomposition (43) over $\mathcal{T}$ gives two expressions of the same quantity, the expected excess risk $\mathbb{E}_{\mathcal{T}}\left[\mathcal{R}_{p_J}(h_{\mathcal{T}})\right] - \mathcal{R}^B_{p_J}$:

$$\text{approximation error} + \mathop{\mathbb{E}}_{\mathcal{T}}\left[\text{estimation error}\right] \;=\; \mathop{\mathbb{E}}_{\mathbf{x}}\left[B(\mathbf{x})\right] + \mathop{\mathbb{E}}_{\mathbf{x}}\left[V(\mathbf{x})\right]. \tag{48}$$

The totals always coincide; the individual terms, in general, do not. The bias compares $\bar{h}$ with h^B, the approximation error compares $h^*_{\mathcal{H}}$ with h^B, and $\bar{h}$ need neither coincide with $h^*_{\mathcal{H}}$ nor even belong to $\mathcal{H}$ (an average of neural networks is not, in general, a neural network of the same architecture).